

Searching

- Linear Search
- Binary Search
 - It is very fast and efficient.
 - It requires the list of elements to be in sorted order.
 - Process:
 - In this method, to search an element we compare it with the element present at the centre of the list. If it matches then the search is successful otherwise, the list is divided into two halves – one from the first element to centre element (first half), another from the centre element to the last element (second half).
 - The searching will now proceed in either of the two halves depending upon whether the target element is greater or smaller than the centre element.
 - The process of comparing the required element with the centre element and if not found then dividing the elements into two halves is repeated till the element is found or further subdivision of array is possible.

Searching & Hashing Unit II. 1 Dr. Rabins Porwal

1

Binary Search Algorithm

BINARY (DATA, LB, UB, ITEM, LOC)
(The algorithm finds the location LOC of ITEM in the array DATA, or sets LOC=NULL if the search is unsuccessful)

- [Initialize segment variables]
Set BEG = LB, END = UB and MID = INT ((BEG+END)/2)
- Repeat steps 3 and 4 while BEG ≤ END and DATA[MID] ≠ ITEM
- If ITEM < DATA[MID] then
set END = MID - 1
else
set BEG = MID + 1
[End of If structure]
- Set MID = INT ((BEG+END)/2)
[End of step 2 loop]
- If DATA[MID] = ITEM, then
set LOC = MID
else
LOC = NULL
[End of If structure]
- Exit

Searching & Hashing Unit II. 2 Dr. Rabins Porwal

2

Complexity of Binary Search

The complexity is measured by the number $f(n)$ of comparisons to locate ITEM in DATA where DATA contains n elements. It can be observed that **each comparison reduces the sample size in half**. Hence if $f(n)$ comparisons are required to locate ITEM, then

$$2^{f(n)} > n \text{ or equivalently } f(n) = \log_2 n + 1$$

That is, the running time for the worst case is approximately equal to $O(\log_2 n)$. It can also be shown that the running time for the average case is approximately equal to the running time for the worst case.

Searching & Hashing Unit II. 3 Dr. Rabins Porwal

3

Example on Complexity of Binary Search Algorithm

Suppose DATA contains 1000000 elements. It can be observed that $2^{10} = 1024 > 1000$ and hence $2^{20} > 1000^2 = 1000000$. Hence, using the binary search algorithm, only about 20 comparisons will be required to find the location of an item in a DATA array with 1000000 elements.

Searching & Hashing Unit II. 4 Dr. Rabins Porwal

4

Limitations of Binary Search Algorithm

- Though this type of search algorithm is very efficient but it has some limitations on arrays:
 - The list must be sorted.
 - One must have direct access to the middle element in any sublist
 - Keeping data in a sorted array is very expensive when there are many insertions and deletions.
- Alternative data structures:
 - Usage of linked list or Binary Search tree to store the data.

Searching & Hashing Unit II. 5 Dr. Rabins Porwal

5

Hashing

- The search time of each algorithm discussed so far depends on the number n of elements in the collection of data. (*Drawbacks of previous search techniques.*)
- It is a technique, used to perform a search which is essentially independent of the number of elements, i.e., n .
- It provides a different mechanism to search a table for a given key value.
- In hashing, the record corresponding to a given key value is directly referred by calculating its **address from key value**.
- Hashing tries to compute the address where an element is to be inserted or found in a table by applying a **hash function** to a given key value.

Searching & Hashing Unit II. 6 Dr. Rabins Porwal

6

Terminology

- First of all, let there be a file F of n records with a set K of keys which uniquely determine records in F .
- Secondly, it is assumed that F is maintained in memory by a table T of m memory locations and that L is the set of memory addresses of the locations in T . (For convenience, it is assumed that the keys in K and the addresses in L are decimal integers.)

Searching & Hashing Unit II. 7 Dr. Rabins Porwal

7

Hash Function

- A function H from the set K of keys into the set L of memory addresses is called a **hash function** or **hashing function**.

$$H : K \rightarrow L$$

- Weakness/ Problem:
 - Unfortunately, such a function H may not yield distinct values: it is possible that two different keys k_1 and k_2 will yield that same hash addresses. This is called **collision**. And there should be some **collision resolution strategy**, if it occurs.

Searching & Hashing Unit II. 8 Dr. Rabins Porwal

8

Hash Function (Contd...)

- While selecting hash function $H : K \rightarrow L$, two principal criteria are used:
 - First, the function H should be very easy and quick to compute.
 - Second, the function H should, as far as possible, uniformly distribute the hash addresses throughout the set L so that there are a minimum number of collisions.
- (The term "hashing" comes from the technique of "chopping" a key into pieces)

Searching & Hashing Unit II. 9 Dr. Rabins Porwal

9

Example

- Suppose a company with 68 employees assigns a 4-digit employee number to each employee which is used as the **primary key** in the company's employee file.
- Alternative method:
 - The employee number can be used as the address of the record in memory. The search will require space for 10000 memory locations, whereas space for fewer than 30 such locations would actually be used. Clearly this tradeoff of space for time is not worth the expense.

Searching & Hashing Unit II. 10 Dr. Rabins Porwal

10

Hash Function (Contd...)

- Some popular hash functions are as follows:
 - Division Method
 - Mid Square Method
 - Folding Method
 - Truncation Method

Searching & Hashing Unit II. 11 Dr. Rabins Porwal

11

Division Method

- In this method, choose a number m (memory locations) larger than the number n of keys in K .
 - The number m is usually chosen to be a prime number or a number without small divisors, since this minimizes the number of collisions.
- The hash function H is defined as follows:

$$H(k) = k \pmod{m} \quad \text{or} \quad H(k) = k \pmod{m} + 1$$
 - Here $k \pmod{m}$ denotes the remainder when k is divided by m .
 - The second formula is used when it is required that the hash addresses should range from 1 to m rather than from 0 to $m-1$.

Searching & Hashing Unit II. 12 Dr. Rabins Porwal

12



Example

- On board

Searching & Hashing Unit II. 13 Dr. Rabins Porwal

13



Mid Square Method

- In this method the key k is squared. The Hash function H is defined by

$$H(k) = I$$
 where I is obtained by deleting digits from both ends of k^2 . Here it is emphasized that the same positions of k^2 must be used for all of the keys.

Searching & Hashing Unit II. 14 Dr. Rabins Porwal

14



Example

- On board

Searching & Hashing Unit II. 15 Dr. Rabins Porwal

15



Folding Method

- In this method, the key k is partitioned into a number of parts, $k_1, k_2, \dots, k_r$, where each part, except possibly the last has the same number of digits as the required address. Then the parts are added together, ignoring the last carry. That is,

$$H(k) = k_1 + k_2 + \dots + k_r$$
 where the leading-digit carries, if any, are ignored.
- **Note:** Sometimes the even-numbered parts $k_2, k_4, \dots$, are reversed before addition.

Searching & Hashing Unit II. 16 Dr. Rabins Porwal

16



Example

- On board

Searching & Hashing Unit II. 17 Dr. Rabins Porwal

17



Truncation Method

- Oldest method for computing hash value.
- In this method, only some parts of the key is taken as address based on the size of hash table, it can be some rightmost digit or leftmost digit.
- **Note:** Here Chance of collision is more because last two digits can be same in different keys.

Searching & Hashing Unit II. 18 Dr. Rabins Porwal

18

Example

- On board

Searching & Hashing Unit II. 19 Dr. Rabins Porwal

19

Collision/ Conflict Resolution Strategy

- While inserting a new record R with key k in a file F , if memory location $H(k)$ is already occupied, then this situation is called **collision** / **conflict**.
- There are two general ways of resolving collision:
 - Open addressing (Closed hashing)
 - Chaining (Separate chaining/ Open hashing)

Searching & Hashing Unit II. 20 Dr. Rabins Porwal

20

Load factor & Probes

- Which of the strategy should be chosen, it depends on many factors.
- One important factor is the ratio of the number n of keys in K (which is the number of records in F) to the number m of hash addresses in L . This ratio, $\lambda = n/m$, is called the **load factor**.
- The efficiency of a hash function with a collision resolution procedure is measured by the average number of probes (key comparisons) needed to find the location of the record with a given key k . The efficiency depends mainly on the load factor λ .

Searching & Hashing Unit II. 21 Dr. Rabins Porwal

21

Open Addressing

- Linear Probing:
 - Suppose that a record R with key k is to be added to the memory table T , but that memory location with hash address $H(k) = h$ is already filled. One natural way to resolve the collision is to assign R to the first available location following $T[h]$. (**Assumption:** The table T with m memory locations is *circular*, i.e., $T[1]$ comes after $T[m]$.)
 - Accordingly, with such a collision procedure, searching for the record R is done in the table T by linearly searching the locations $T[h]$, $T[h+1]$, $T[h+2]$, ... until finding R or meeting an empty location, which indicates an unsuccessful search.
 - This procedure is called **linear probing**.

Searching & Hashing Unit II. 22 Dr. Rabins Porwal

22

Example

- On board

Searching & Hashing Unit II. 23 Dr. Rabins Porwal

23

Open Addressing (Contd...)

- Disadvantages of Linear Probing:
 - The main disadvantage of this technique is primary **clustering problem**. When half of the table is full, then it is difficult to find empty position in hash table in case of collision. The records tend to cluster together. Searching will also become slow because it will go for sequential searching.
- Quadratic Probing
 - In this technique locations are linearly searched with addresses using quadratic increments, i.e., h , $h+1$, $h+4$, $h+9$, ..., $h+P^2$, ...

Searching & Hashing Unit II. 24 Dr. Rabins Porwal

24



Example

- On board

Searching & Hashing Unit II. 25 Dr. Rabins Porwal

25



Open Addressing (Contd...)

- Disadvantage of Quadratic Probing:
 - This technique can not search all the locations. If the number m of locations in the table T is a prime number, then the above sequence will access half of the locations in T .

Searching & Hashing Unit II. 26 Dr. Rabins Porwal

26



Open Addressing (Contd...)

- Double Hashing:
 - In this technique, a second hash function is used for resolving a collision.
 - The first hash function is used as a primary function. In case the address determined by the first hash function is occupied, the second hash function is used to increment the address obtained by the first hash function.
 - Suppose a record R with key k has the hash addresses $H(k) = h$ and $H'(k) = h' \neq m$. Then the locations are searched linearly with addresses $h, h+h', h+2h', h+3h', \dots$

Searching & Hashing Unit II. 27 Dr. Rabins Porwal

27



Example

- On board

Searching & Hashing Unit II. 28 Dr. Rabins Porwal

28



Open Addressing (Contd...)

- Disadvantage of Double Hashing:
 - It requires calculation of hash function two times, which makes it complex and thus searching becomes slower than linear & quadratic probing.
- In all open addressing methods the major disadvantage is in the deletion of any key. If there are many keys, which are hashed to the same address then the removal of a key from middle implies that all other keys must be packed, which is time consuming.

Searching & Hashing Unit II. 29 Dr. Rabins Porwal

29



Chaining/ Separate Chaining

- This method maintains the chain of elements which have same hash addresses. Hash table can be taken as an array of pointers. Size of hash table can be number of records. Each pointer points to one linked list and the elements which have same hash addresses are maintained in the linked list.
- It involves two operations:
 - Creation of good hash function for getting hash key value in the hash table.
 - Maintaining the elements in the linked list which is pointed by pointer available in table.

Searching & Hashing Unit II. 30 Dr. Rabins Porwal

30



Chaining (Contd...)



- Advantages:
 - Insertion is very easy.
 - Savings of memory space and perfect collision resolution if the file of records is large and no wastage of memory space.
 - Use of good hash function distributes elements evenly.
- Disadvantages:
 - Wastage of memory space when records are very small.

Searching & HashingUnit II, 31Dr. Rabins Porwal

31